

PALESTRA

Engenharia de IA e Desenvolvimento de Software Orientado a Especificação

Do modelo à spec: como times de software crítico incorporam IA sem perder auditabilidade.

120 minutos · desenvolvedores sêniores · sistemas de controle de infecção hospitalar (IRAS)

Agenda: doze blocos, três pilares

Bloco	Tema	min
0	Abertura e agenda	3
1	Linha do tempo: do Perceptron aos agentes	3,5
2	MODELOS DE LINGUAGEM (LLMs) — arquitetura e panorama	14
3	O que mudou para quem escreve software	2
4	Engenharia de contexto e RAG	8
5	Agentes: anatomia, ferramentas, padrões, falhas	14
6	Protocolos e formatos de interoperabilidade	7,5
9	DESENVOLVIMENTO ORIENTADO A ESPECIFICAÇÃO (SDD)	19
11	OS SISTEMAS: LangNet, demonstração, AI Co-	15 5

① LLMs

o motor: o que o modelo é e o que não é

② SDD

o método: especificar e verificar

③ Sistemas

a prova: nossos sistemas rodando

A tese central da palestra

“O gargalo deixou de ser escrever código. Passou a ser especificar e verificar.”

Capacidade virou mercadoria comum (commodity). Qualquer um chama uma API e gera código; o que separa os times é o método — não o acesso ao modelo.

Agente sem portão de verificação degrada de forma previsível. Sem um “gate” (portão que confere cada passo), a taxa de erro cai de um jeito que a matemática prevê — mostramos a curva no S28.

Especificação executável é o que torna código de IA auditável. Condição inegociável em software de saúde: sem spec verificável, não há como provar conformidade.

Do Perceptron ao desenvolvimento agêntico

ARQUITETURAS



"a mesma arquitetura, só que maior"

O QUE O DESENVOLVEDOR FAZIA



A ideia é de 1958. O que mudou foi compute e dado — e a unidade de trabalho de quem escreve software.

IA na saúde (1/3) — Prognóstico e avaliação de risco

Prognóstico = **prever o risco de um evento futuro** (adoecer, agravar, morrer). A IA já fazia isto sobre tabelas; agora, lendo o **texto livre do prontuário**, passa a fazê-lo sobre a narrativa clínica.

Algoritmo (nome por extenso)	O que faz — para que serve	Onde entra na infectologia (IRAS)
Escore de Risco	Soma ponderada dos fatores do paciente e devolve um número que estima o risco.	Risco de infecção por paciente (sítio cirúrgico, corrente sanguínea) a partir de dispositivos, tempo de internação e comorbidades.
Regressão Logística	Traça a fronteira que separa 'o evento vai ocorrer' de 'não vai'.	Classificar quem tem alto risco de infectar e priorizar a vigilância ativa.
Análise de Sobrevivência	Modela o tempo até o evento e o risco instantâneo (função de perigo) ao longo do tempo.	Tempo até a infecção associada a dispositivo (por dia de cateter, sonda ou ventilador).
Modelo de Perigos Proporcionais de Cox	Mede quanto cada fator multiplica o risco (ex.: fumar aumenta o risco em 9%).	Quanto cada fator (dias de dispositivo, UTI, idade) multiplica o risco — alvo dos pacotes de prevenção.
Índice de Concordância (C-Index)	Avalia se o escore de risco discrimina bem (0,5 = acaso; 1,0 = perfeito).	Validar se o escore de risco de infecção realmente separa quem vai infectar de quem não vai.

IA na saúde (2/3) — Diagnóstico de doenças

Diagnóstico = **dizer se a doença está presente agora** — um problema de classificação, sobre imagens ou sobre exames.

Algoritmo (nome por extenso)	O que faz — para que serve	Onde entra na infectologia (IRAS)
Rede Neural Convolucional	Aprende padrões em imagens e classifica (ex.: COVID-19 em tomografia; achados em raio-X).	Ler raio-X / tomografia de pneumonia associada à ventilação; triagem automática de imagem.
Rede U-Net	Segmenta a imagem — delimita e mede a lesão ou a estrutura de interesse.	Localizar e medir consolidações pulmonares; segmentar o achado radiológico para acompanhar.
Florestas Aleatórias	Combina muitas árvores de decisão para classificar a partir de exames em tabela.	Prever multirresistência a partir do perfil do paciente e da microbiologia.
Máquinas de Vetores de Suporte	Separa as classes pela melhor fronteira possível entre os dados.	Classificar culturas (positiva/negativa, resistente/sensível) a partir dos exames.
K-Vizinhos Mais Próximos	Classifica um caso pelo que é mais parecido entre os casos vizinhos conhecidos.	Sugerir o patógeno provável por semelhança com casos anteriores da própria base.
Árvore de Decisão	Encadeia perguntas sobre os exames até chegar a uma conclusão — regras legíveis.	Materializar as definições de caso (encadeamento de critérios) de forma auditável.

IA na saúde (3/3) — Tratamento de doenças

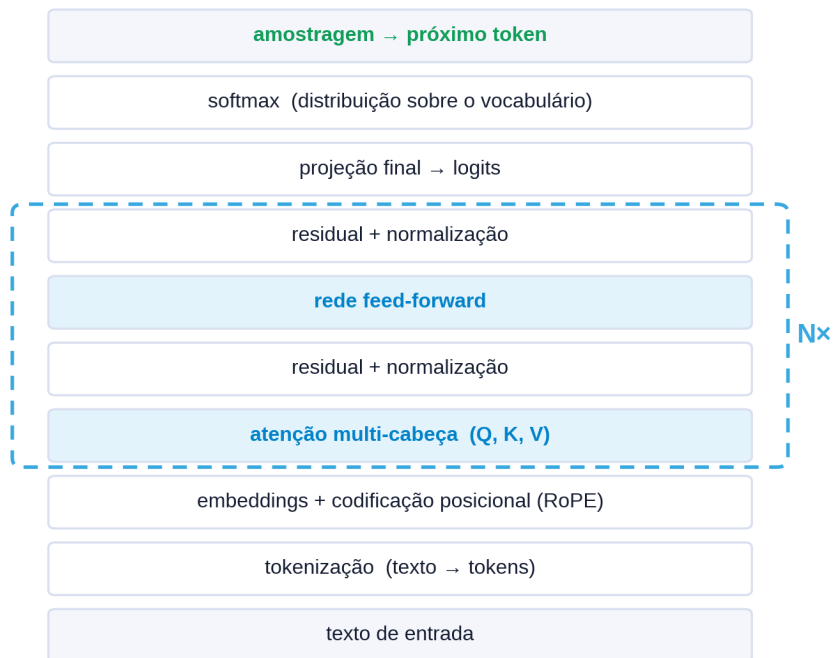
Tratamento = **medir o efeito real de uma intervenção** — o terreno da inferência causal, e não só da correlação.

Algoritmo / método (nome por extenso)	O que faz — para que serve	Onde entra na infectologia (IRAS)
Ensaio Clínico Randomizado	Aloca pacientes por sorteio em tratamento e controle e compara os desfechos.	Provar, sem viés, se um pacote de prevenção ou um protocolo reduz a taxa de infecção.
Redução de Risco Absoluta	Diferença entre o risco de quem recebeu o tratamento e o de quem não recebeu.	Quanto um bundle ou uma precaução reduz, em pontos, a taxa de infecção.
Inferência Causal (Modelo Causal de Neyman-Rubin)	Estima o efeito real de tratar comparando o resultado com e sem o tratamento.	Efeito real do controle de antimicrobianos (stewardship) sobre a resistência, controlando confundidores.
Efeito Médio do Tratamento	A média, na população, da diferença entre tratar e não tratar (ex.: vacina reduziu o risco em 0,2).	Efeito médio de uma política de higienização ou isolamento sobre a incidência de IRAS.

Estes algoritmos já existem; o que muda no **software de vocês** é rodá-los sobre o texto livre e mantê-los **auditáveis e rastreáveis** — o diferencial em saúde.

Anatomia do Transformer

fluxo de baixo para cima — previsão do próximo token



Q, K, V (consulta, chave, valor). Cada token faz uma pergunta; todos anunciam o que têm; a semelhança decide de quem ele copia informação.

Multi-cabeça. Várias relações capturadas em paralelo, cada cabeça olhando um aspecto diferente.

O ponto de engenharia: a atenção é $O(n^2)$. Custo por token, tamanho da janela e toda a economia de contexto nascem dessa quadrática.

Por que o Transformer venceu

Recorrência (LSTM) é sequencial no treino. Uma palavra depois da outra — não aproveita a placa de vídeo.

Atenção é paralelizável. Todos os tokens de uma vez, na GPU. Foi isso que mudou o jogo.

Não venceu por qualidade por token — venceu por VAZÃO de treinamento. Deu para treinar modelos muito maiores em tempo viável. Escala, não elegância.

A arquitetura de hoje: decoder-only, autorregressiva, objetivo único. Um só alvo de treino: prever o próximo token.

Escala e o que emerge dela

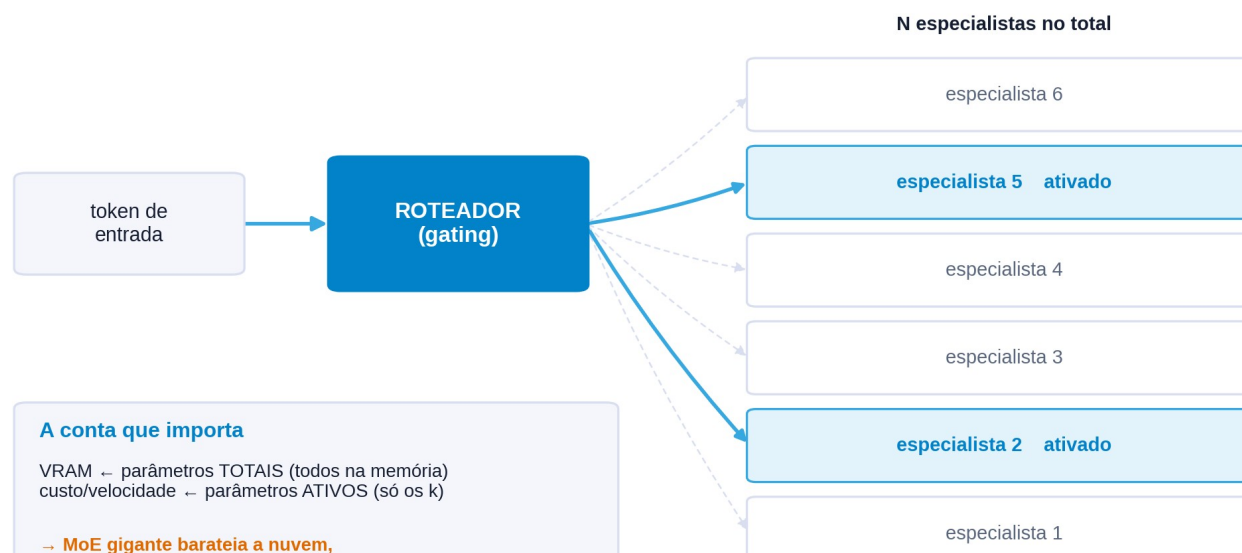
Leis de escala. Mais dados + mais parâmetros + mais processamento → menos erro, de forma previsível. Foi o que justificou o investimento.

O salto de 2020: aprendizado no contexto (in-context learning). O modelo aprende a tarefa na hora da resposta, pelos exemplos do próprio prompt, sem tocar nos pesos.

Consequência: prompt e RAG passam a funcionar. E o ajuste fino (fine-tuning) deixou de ser o caminho obrigatório.

Deslocamento recente: do processamento no TREINO para o processamento na RESPOSTA. O modelo “pensa mais” na hora — volta idêntico no AI Co-Scientist (S62).

MoE: mistura de especialistas



MoE = *Mixture of Experts* (mistura de especialistas). Ex.: DeepSeek-V4-Pro tem 1,6 trilhão de parâmetros totais, mas só 49 bilhões ativos por token.

Inferência: os parâmetros que importam

Temperatura e top-p. Controlam se a amostragem é mais criativa ou mais determinística.

Cache de chaves-valores (KV cache) e cache de prefixo. Reaproveitam contas já feitas — menos custo, menos espera.

Quantização. Reduz a precisão dos pesos (ex.: 4 bits) para caber em menos memória de vídeo.

Regra prática: código e extração estruturada pedem temperatura baixa; exploração de hipóteses pede o contrário — gancho para o S62.



```
# a MESMA chamada, dois resultados

resp = modelo.gerar(prompt,
    temperatura=0.0)
# código / extração: baixa

resp = modelo.gerar(prompt,
    temperatura=1.0)
# exploração: alta
```

Panorama: modelos proprietários

Modelo	Fabricante	SWE-bench Verified	Fonte da medição
GPT-5.6 Sol	OpenAI	~96%	Vals AI (independente)
Claude Fable 5	Anthropic	~95%	medição independente
Gemini 3.1 Pro	Google	agrupamento superior	—

Ressalva obrigatória: o número do fabricante e o de uma medição independente divergem, às vezes muito. Cite sempre a fonte.

SWE-bench Verified = teste de correção de bugs reais de software (%% de tarefas resolvidas). Última verificação: __ / __ / ____

Panorama: modelos abertos para código

Modelo	Arquitetura	Contexto	Licença	Onde roda
Kimi K3	2,8T / 104B ativos	1M	pesos abertos	infra séria
DeepSeek-V4-Pro	1,6T / 49B ativos	1M	MIT	cluster
DeepSeek-V4-Flash	284B / 13B ativos	1M	MIT	servidor médio
GLM-5.2	MoE	longo	MIT	servidor
Qwen3.8-27B	denso 27,8B · multimodal	262K → 1M	Apache-2.0	24 GB VRAM
Qwen3.6-27B	denso 27B	longo	Apache-2.0	1 GPU consumo
Muse Glimmer 30B	denso · multimodal	131K	Apache-2.0	24 GB quantiz.
MiniMax M3	428B / 23B ativos	1M	comunitária ⚠	ver licença

O caso que importa para vocês: Qwen3.8-27B — denso, multimodal, Apache-2.0, roda em 24 GB e, em SWE-bench Pro, supera o Claude Opus 4.6 Max.

Alerta jurídico: “pesos abertos” ≠ “código aberto”. MIT e Apache-2.0 são livres; a licença do MiniMax é não comercial por padrão. · Última verificação: __/__/____

Contexto longo: o número anunciado é o menos útil

Anúncios de 1 a 10 milhões de tokens. Vários modelos anunciam janelas gigantes — sem um teste publicado que sustente qualidade perto disso.

Degradação do contexto (“context rot”). A capacidade efetiva fica em 60–70% do anunciado; e a queda não é gradual — segura até um limiar e despenca. Uma janela de 1M pode degradar já em 50 mil.

Resultado contraintuitivo (testes da Chroma). Os modelos foram melhores com texto embaralhado do que coerente: texto coerente cria viés de recência — sobrepeça o fim e esquece o começo.

Custo. Encher 1 milhão de tokens vai de centavos a dezenas de dólares — diferença de dezenas de vezes entre modelos.

Contexto menor e curado supera contexto grande e velho. Pior em domínio regulado: quando a janela estoura, o modelo corta em silêncio — sem registro do que se perdeu.

A escada de abstração

Assembly → **linguagem de alto nível**. Objeção da época: “vai gerar código lento e sem controle”. Consolidou-se com o COMPILADOR.

Alto nível → **frameworks**. Objeção: “esconde o que importa, vira mágica”. Consolidou-se com o VERIFICADOR DE TIPOS e os TESTES.

Frameworks → **especificação + verificação**. É o degrau de agora. A mesma objeção — “não dá para confiar” — e a mesma resposta: uma verificação nova.

O padrão nunca muda. Cada degrau sobe o nível do que você escreve — e só se firma quando ganha a sua camada de verificação automática.

O que a IA faz bem e mal em código

FAZ BEM

- código de fronteira **bem especificado**
- traduzir entre representações
- gerar **testes a partir de critérios explícitos**
- refatoração mecânica

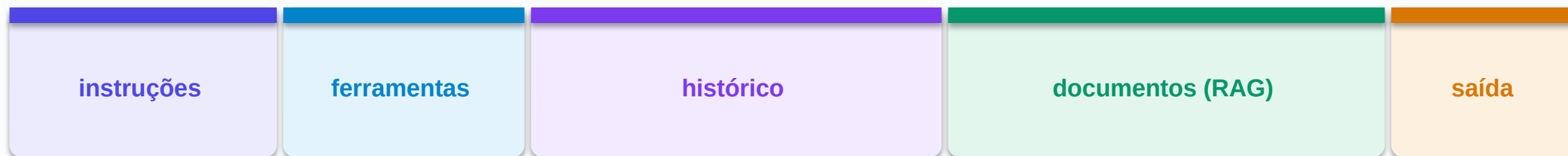
FAZ MAL

- decisão arquitetural com **trade-off implícito**
- corretude que depende de **contexto não escrito**
- invariantes **não declarados**
- regras que “todo mundo sabe” e ninguém anotou

A coluna da direita é toda ela **falta de especificação**. Custo típico: 50 mil a 500 mil tokens por tarefa; o português gasta ~1,5× mais tokens que o inglês.

De “prompt” para “engenharia de contexto”

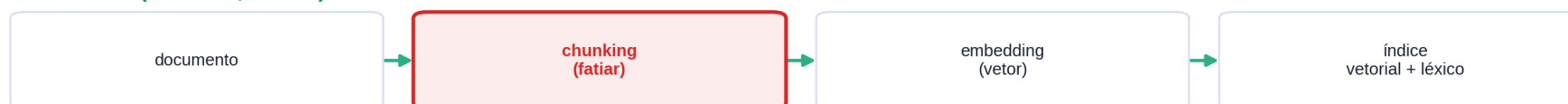
O trabalho não é mais escrever uma frase esperta. É **decidir o que ocupa a janela** — o espaço de atenção é finito e caro (lembre do $O(n^2)$ e do S13).



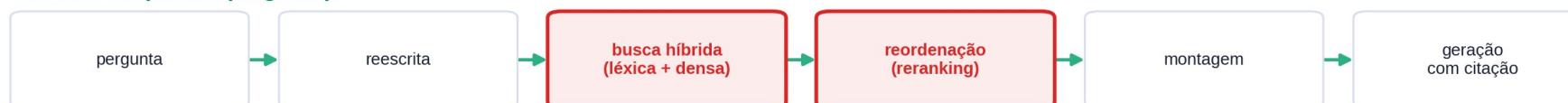
A “janela de contexto”: um orçamento fixo a ser gasto com disciplina.

RAG: como funciona, e onde quebra

INGESTÃO (uma vez, offline)



CONSULTA (a cada pergunta)



Os três pontos de quebra: fatiar mal, recuperar o trecho errado, e a ausência de reordenação.

RAG raramente falha na geração — falha na recuperação, e o time gasta semanas ajustando o prompt.

RAG = Retrieval-Augmented Generation (geração aumentada por recuperação).

Por que fatiar mal destrói um critério de IRAS

O critério de definição de caso (ICSAC) ocupa três parágrafos. Critério clínico + critério laboratorial + janela temporal — os três precisam ser lidos JUNTOS.

Em pedaços de 512 tokens, o laboratorial se separa da janela. A recuperação devolve metade da regra; o modelo completa o resto sozinho — e notifica um FALSO POSITIVO.

A correção: fatiar pela unidade lógica, não por tamanho fixo. Chunking semântico pela regra inteira + metadados por tipo de infecção + expansão de janela ao redor do trecho.

Saída estruturada como contrato

```
{  
  "criterio": "ICSAC-corrente-sanguinea",  
  "atendido": true,  
  "evidencia_textual":  
    "hemocultura + para S. aureus em 12/03",  
  "data_referencia": "2026-03-12",  
  "confianca": 0.91  
}
```

Um esquema (schema) por critério. Booleano do resultado, o trecho literal que o justifica, a data de referência e a confiança.

Todo booleano vem com a evidência citada. Sem o trecho literal não há revisão humana viável nem trilha de auditoria.

Não peça parecer. Peça formulário preenchido. “Formulário com evidência citada”, não “texto livre opinando”.

Avaliação e proteções (guardrails)

Eixo	O que medir	Por quê
Estratificação	sensibilidade e especificidade POR tipo de infecção	o custo do erro muda por infecção
Conjunto dourado	casos revisados por infectologista, versionado	verdade de referência estável
Gatilho de CI	roda a cada mudança de prompt, MODELO ou base	trocar versão = mudança controlada
Asserção	um teste com assert sobre o conjunto dourado	o portão é código, não opinião

Acurácia agregada é inútil aqui: falso negativo e falso positivo têm custos assimétricos e diferentes por infecção. Guardrails: menor privilégio, anonimizar antes de sair, e cuidado com injeção vinda do próprio prontuário (LGPD).

Definição operacional (sem misticismo)

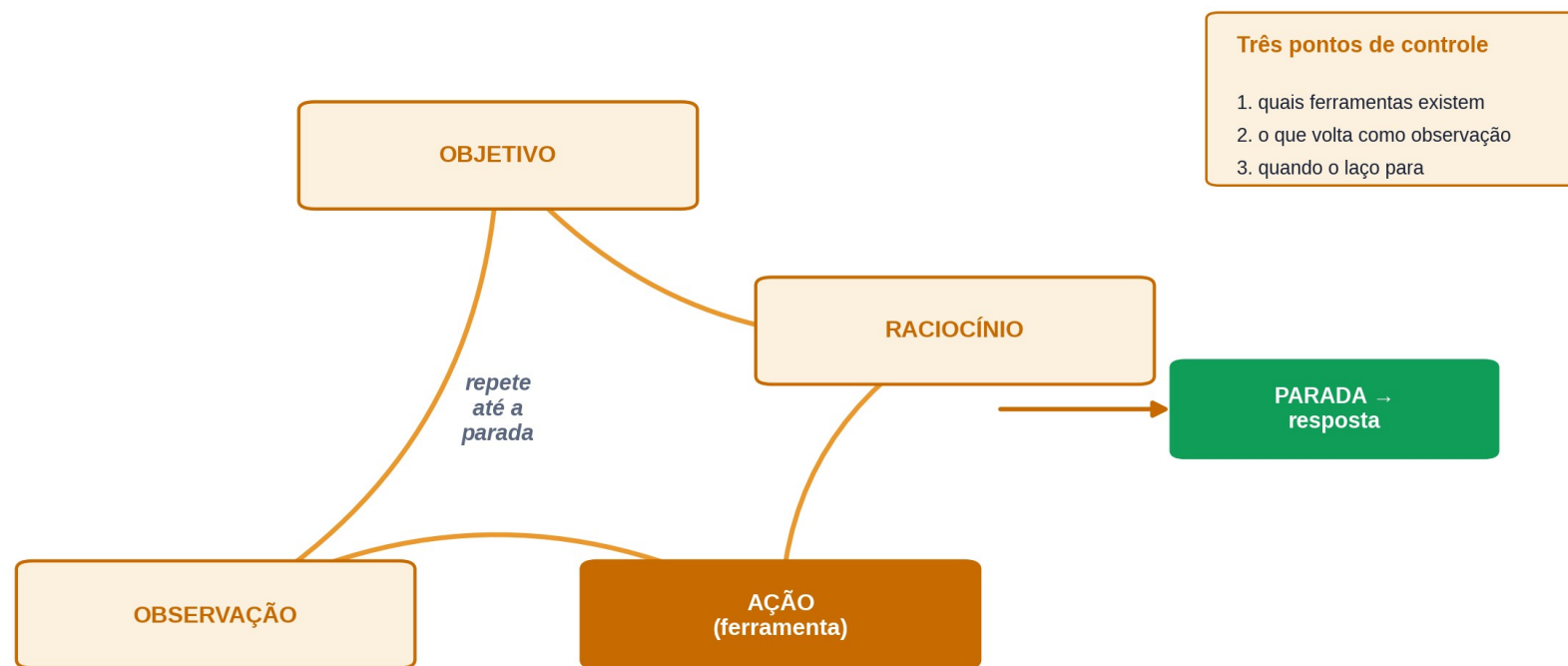
Um agente é um modelo em laço, com ferramentas, sobre um ambiente, com critério de parada.

O que **NÃO** é agente: fluxo de caminho fixo com o modelo nos nós. Isso é um workflow — e quase sempre é a escolha certa.



Eixo da autonomia: da esquerda para a direita, mais poder e mais risco. Autonomia é custo, não virtude — escolha o mínimo que resolve.

O laço agêntico



Um agente = um modelo em laço, com ferramentas, sobre um ambiente, com critério de parada.

“Tool use”: usar ferramentas sem framework nenhum



```
# 1. você DESCREVE a ferramenta (isto é prompt!)
ferramenta = {
    "nome": "consultar_microbiologia",
    "parametros": {"paciente_id": "str",
                   "janela_dias": "int"}
}
# 2. o modelo responde pedindo a chamada (tool_use)
# 3. VOCÊ executa e devolve o resultado (tool_result)
# 4. repete num while explícito até a resposta
```

O laço agêntico tem trinta linhas. Tudo que vem depois — os frameworks do Bloco 7 — é conveniência, não mágica.

A descrição da ferramenta É prompt.

Descrição ruim de ferramenta causa mais falha do que prompt de sistema ruim.

Componentes de um agente

Memória curta × longa. Curta = a conversa atual; longa = o que persiste entre execuções.

Planejamento. Quebrar o objetivo em passos antes de agir.

Reflexão. Criticar o próprio resultado e tentar de novo.

Memória longa em arquivo versionado costuma superar a vetorial em regras estáveis. É auditável e comparável em diff — gancho direto para o formato OKF do S31.

Os seis padrões de composição



Comece pelo mais simples que resolve. Os mais subutilizados e úteis aqui: VOTAÇÃO e AVALIADOR-OTIMIZADOR.

O mesmo caso resolvido de três formas

Caso: agente de vigilância de IRAS — varre microbiologia, prescrição e evolução; aplica os critérios; propõe a notificação; e PARA em revisão humana.

Encadeado

passos fixos em sequência; simples, mas rígido

Orquestrador-trabalhadores

um trabalhador por tipo de infecção, contexto isolado

Avaliador-otimizador

um crítico contesta contra o critério normativo: menos falso positivo, ~2× tokens

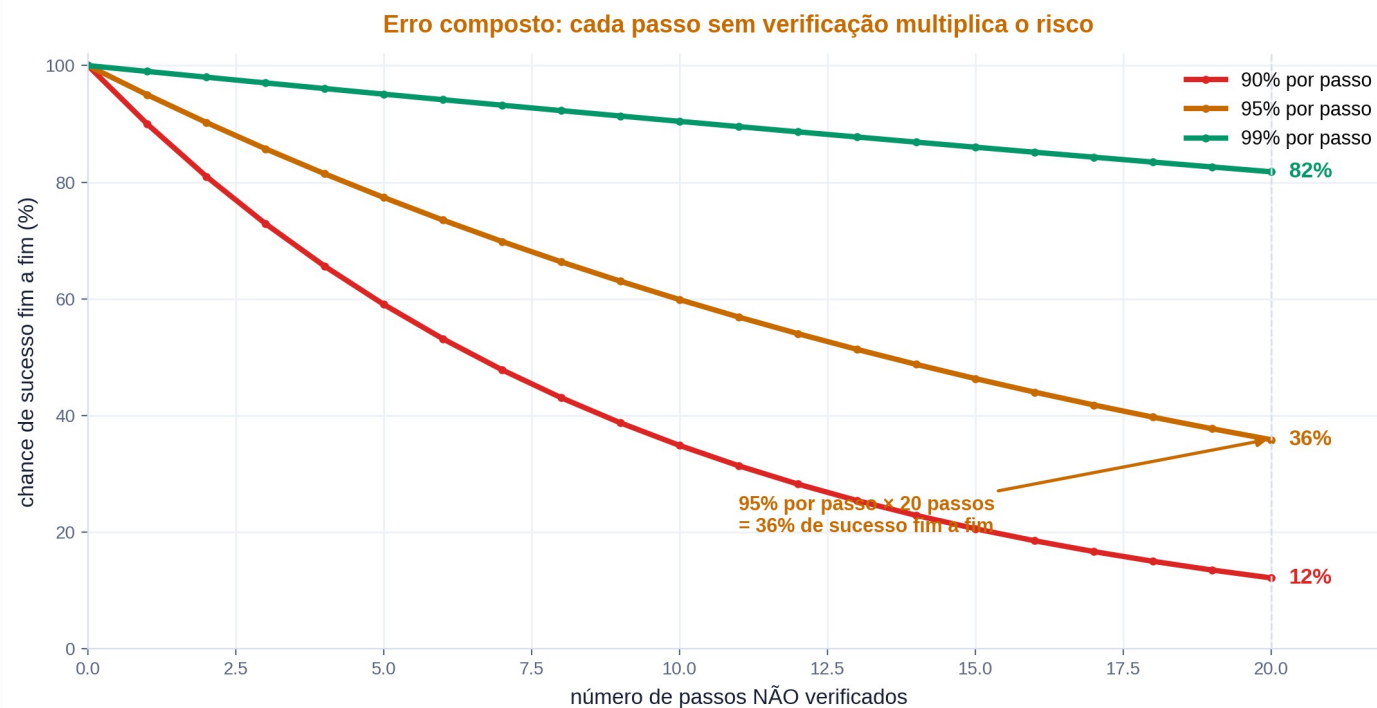
Multiagente: quando compensa (e quando não)

Só compensa com subtarefas paralelizáveis E contextos isolados. Se as partes não são independentes, o custo explode sem ganho.

Em fluxo linear: 4 a 15× mais tokens para reimplementar um “if”. Vários agentes conversando para decidir o que um condicional resolveria.

A passagem de contexto entre agentes é “lossy”. Cada repasse perde informação — como fotocópia de fotocópia.

Por que agentes falham em produção

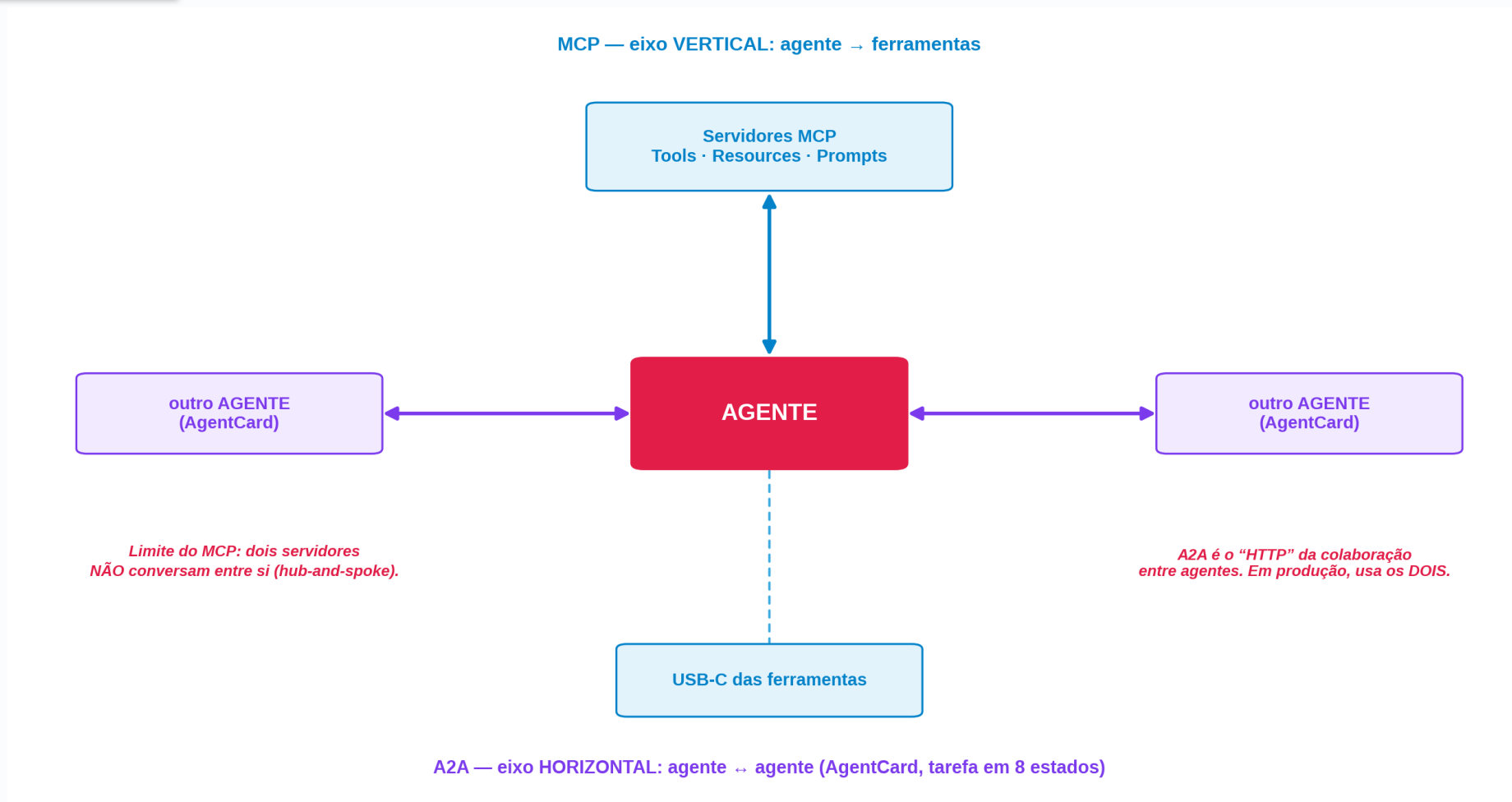


Erro composto. 95% de acerto por passo, 20 passos → só 36% de sucesso fim a fim.

Modos de falha: laço infinito, ferramenta alucinada, contexto contaminado, custo imprevisto — e o pior, a FALHA SILENCIOSA.

A saída não é “prompt melhor”. É reduzir o número de passos NÃO verificados. Guardem esta frase para o Bloco 9.

O mapa: vertical (MCP) × horizontal (A2A)



A família A2A e as alternativas

Extensões oficiais do A2A. Secure Passport (identidade), Timestamp, Traceability (rastreabilidade), Agent Gateway.

A família: AP2, UCP, A2UI. Pagamentos iniciados por agente, comércio com consentimento criptográfico, e interface.

Alternativas: ACP (IBM) e ANP. ACP herda a tradição FIPA-ACL com performativas tipadas; ANP aposta em identidade descentralizada.

A crítica que dá valor ao slide: nenhum protocolo expressa GOVERNANÇA. Registram quem chamou quem — não sob qual política nem com que base legal. Em saúde, essa camada sobra para VOCÊS.

OKF: um formato aberto de conhecimento

① Um pacote (bundle) é um diretório de arquivos markdown. Nada de banco de dados, nada de servidor.

② Cada arquivo é um conceito; o caminho do arquivo é o identificador.

③ Cada arquivo abre com frontmatter YAML — único campo obrigatório: type.

④ Arquivos se referenciam por links markdown comuns. O que transforma o diretório num GRAFO, não numa lista.

⑤ Dois nomes reservados: index.md (listagem) e log.md (histórico).

Para vocês, direto:

um pacote OKF com as definições de caso de IRAS, as fórmulas dos indicadores e os runbooks da CCIH.

Legível por agente, revisável por infectologista, com log.md de histórico e “git blame” de autoria —

metade do que uma auditoria já pede.

Ressalva: é v0.1 — padroniza o contêiner, não a semântica clínica.

Os outros formatos em markdown

AGENTS.md. Contexto do projeto para agentes de código: comandos de build e teste, convenções, decisões. Markdown puro.

DESIGN.md. Identidade visual: tokens de design em YAML mais o racional em prosa.

Agent Skills. Capacidade empacotada como uma pasta com um arquivo markdown, com registros públicos.

O padrão por trás dos três: markdown versionado no repositório. A única coisa que humano e máquina leem, que o git versiona e que o revisor audita num diff.

A ponte para o SDD

Especificação (SDD)

o QUE o sistema deve fazer

AGENTS.md

COMO se trabalha no repositório

Pacote OKF

o DOMÍNIO que ele modela

São a MESMA ideia em escopos diferentes: conhecimento formalizado em markdown versionado, consumido por agente e revisável por humano. Os três convivem no mesmo repositório — e é isso que constrói a rastreabilidade (guardem para o S49).

INTERVALO

5 minutos

Na volta: frameworks, ambientes agênticos e o núcleo — o SDD.

Critério de leitura para os cinco

A pergunta única para cada framework: o que ele controla por mim — o laço, o estado, ou nenhum dos dois?

O mesmo agente do S26 aparece nos cinco. Para comparar o custo da abstração — não a sintaxe.

Frameworks não são mágica; são conveniência sobre o laço de 30 linhas do S23. A pergunta certa não é “qual é mais bonito”, é “o que ele assume por você e o que ele esconde”.

OpenAI Agents SDK

Peças: Agent, handoffs, guardrails, sessions. Repasse entre agentes (handoff), proteções e sessões, com rastreamento embutido.

Modelo de controle: o SDK controla o laço; o estado vive na sessão.

Rastreamento (tracing) de primeira linha — porém acoplado ao provedor. Ótima observabilidade, ao custo de amarrar você à OpenAI.

```
# triagem -> especialista por infecção
triagem = Agent(
    name="triagem",
    handoffs=[uti_agent,
               cirurgia_agent])
# o SDK cuida do laço e do repasse
```

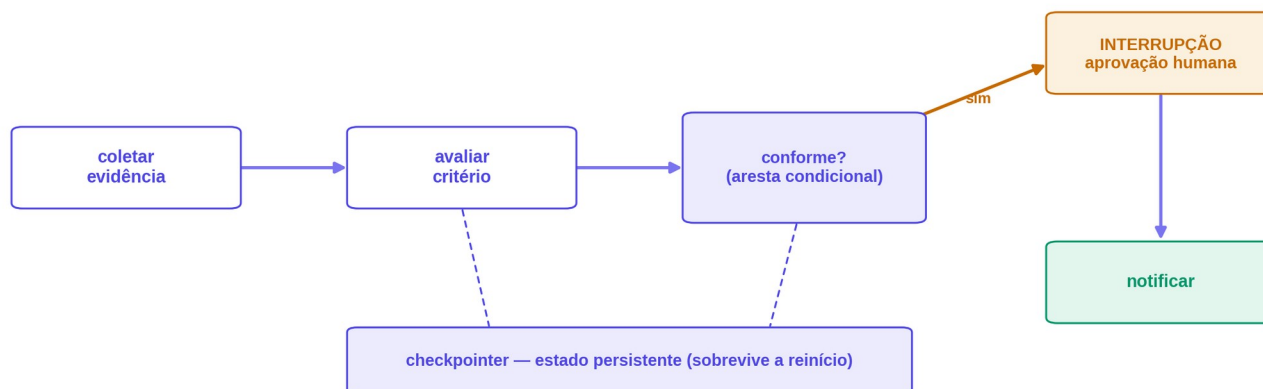
Anthropic Claude Agent SDK

O laço do Claude Code exposto como biblioteca. Busca agêntica no sistema de arquivos SEM indexação prévia, subagentes com contexto isolado, hooks.

Sistema de permissões + MCP nativo + modo headless. Controle fino do que o agente pode fazer, e execução sem interface para rodar em CI.

Diferencial: projetado em torno de CÓDIGO e ARQUIVOS, não de conversa. É o mais relevante para o Bloco 9 (o SDD roda sobre arquivos versionados).

LangChain e LangGraph



LangGraph = grafo de estados: nós, arestas condicionais, estado durável e PARADA para aprovação humana.

LangChain compõe componentes;
LangGraph é máquina de estados.

Estado durável (checkpointer) + parada
para aprovação (interrupt). Nativos — e
decisivos aqui.

Para IRAS: a notificação NÃO sai sem
aprovação, e o processo sobrevive a
reinício.

CrewAI

Modelo mental: organização, não grafo. Agent com papel, objetivo e história; Task com saída esperada; Crew reunindo; Process sequencial ou hierárquico.

Onde brilha: protótipo e demonstração. A menor curva de entrada; decompor por papel força o time a escrever o que cada agente faz — meio caminho para uma spec.

Onde dói: a metáfora de papéis ESCONDE o laço. Quando falha, você depura uma abstração, não uma sequência de chamadas. Estado e durabilidade fracos vs. LangGraph.

Veredito: ótimo para levantar um piloto numa tarde. Pense duas vezes antes de pôr em produção regulada. (É o framework do nosso LangNet — Bloco 11.)

AutoGen / AG2

Modelo mental: conversa entre agentes. ConversableAgent como unidade; GroupChat com um gerente que escolhe quem fala; executor de código embutido.

Humano-no-laço nativo e granular. O modo de intervenção é configurável por agente (sempre, nunca, só ao terminar) — direto no gate de notificação de vocês.

Onde brilha: exploração e pesquisa, caminho não conhecido de antemão. É o parente mais próximo do AI Co-Scientist (S62).

Onde dói: comportamento emergente — a conversa pode divergir e gastar tokens sem convergir. E cuidado com a linhagem: Microsoft × fork comunitário AG2, exemplos incompatíveis.

Comparativo e a opinião contrária

Framework	Controle do laço	Estado	Humano-no-laço	Caso ideal
OpenAI SDK	controla o laço	sessão	guardrails	produtos OpenAI
Claude Agent SDK	controla o laço	arquivos	hooks + permissões	código / arquivos
LangGraph	máquina de estados	durável	nativo (interrupt)	produção regulada
CrewAI	esconde o laço	fraco	limitado	protótipo rápido
AutoGen / AG2	conversa	fraco	granular	exploração / pesquisa
laço próprio	você controla	você faz	você faz	70% dos casos

A opinião contrária: para uns 70% dos casos, o laço direto é mais simples de depurar. Framework se justifica por durabilidade, observabilidade e humano-no-laço — não por elegância. “Não adotem framework antes de ter o eval do S20.”

Mudou a unidade de trabalho

Do autocompletar de uma linha para a tarefa sobre o repositório inteiro.

A revisão passa a ser de um diff inteiro, não de uma sugestão de linha. Isso muda o code review do time — e é uma mudança de processo, não de ferramenta.

Claude Code

CLI com busca agêntica sem indexação prévia. Explora o repositório na hora, em vez de manter um índice que envelhece.

CLAUDE.md, subagentes, skills, MCP, modo headless para CI. O contexto do projeto e as capacidades ficam versionados no repositório.

Hooks são o mecanismo de PORTÃO. Lint, teste e política rodam DENTRO do laço — não depois. É o gate do Bloco 9 acontecendo na prática.

Cursor e o panorama

Cursor: IDE com indexação prévia, Composer/Agent, regras de projeto. Indexa antes (rápido, mas o índice pode ficar defasado).

O trade-off de fundo: indexação prévia (rápida, risco de índice velho) × exploração sob demanda (lenta, sempre atual).

O resto do panorama, numa linha: Copilot, Windsurf, Aider, Codex, Devin — variações do mesmo tema.

O problema: “vibe coding”

Programar “no feeling” funciona no protótipo e colapsa no sistema. Enquanto é pequeno, tudo bem. Quando cresce, ninguém mais segura na cabeça.

O sintoma: o código existe, funciona — e ninguém sabe qual requisito ele atende, nem se ainda atende.

Em domínio regulado, isso tem outro nome: NÃO CONFORMIDADE. Não é dívida técnica; é risco regulatório.

A inversão

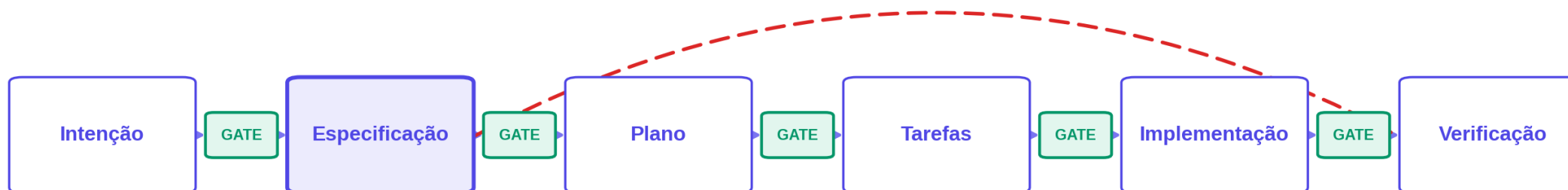
(a) O jeito de hoje: requisito informal → código → documentação que já nasce desatualizada.

(b) O SDD: especificação PRIMÁRIA → plano → tarefas → código DERIVADO.

A spec é o que se versiona, revisa e mantém. O código é o que se regenera. *“É a relação entre código-fonte e binário — ninguém revisa o binário.”*

O ciclo SDD

Cada GATE corta a cadeia não verificada: aprovação humana · teste automatizado · verificação de política · verificação formal



em caso de falha, volta — nenhum passo avança sem passar no seu gate

SDD = Spec-Driven Development (Desenvolvimento Orientado a Especificação). A spec é primária; o código é derivado.

Anatomia de uma especificação útil

Contexto e escopo · NÃO-objetivos · requisitos em EARS. Dizer o que está fora é tão importante quanto o que está dentro.

Critérios de aceitação verificáveis · contratos · invariantes · referências normativas.

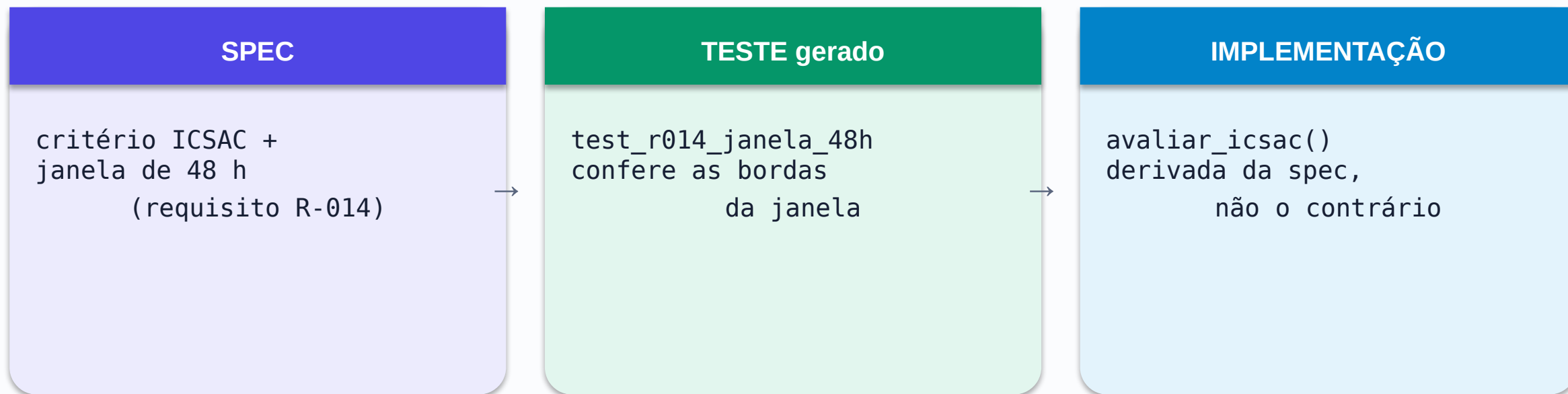
Requisito em EARS (bom):

“Quando uma hemocultura positiva for registrada, o sistema deve avaliar os critérios de ICSAC dentro de 48 h e produzir um parecer com evidência citada.”

Requisito ruim (não verificável):

“O sistema deve detectar infecções corretamente.”

Exemplo ponta a ponta: da spec ao código



Rastreabilidade: R-014 → test_r014_janela_48h → avaliar_icsac(). O teste nasceu do CRITÉRIO, não do código — por isso não herda os bugs da implementação.

Ferramental de SDD

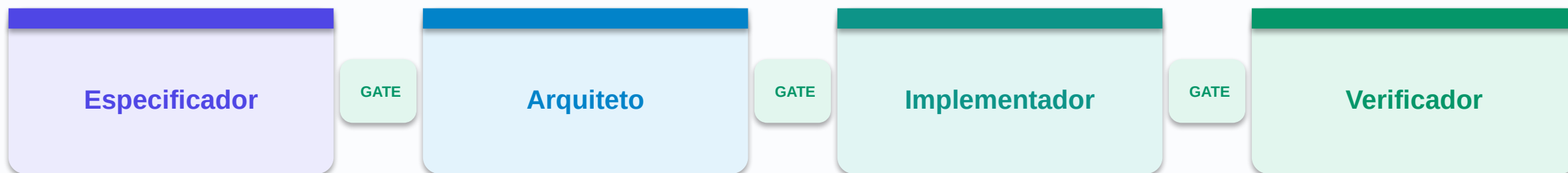
GitHub Spec Kit. Comandos `/specify`, `/plan`, `/tasks`, `/implement` — o ciclo do S46 como fluxo de trabalho.

Kiro. Organiza `requirements.md`, `design.md` e `tasks.md` no repositório.

Tessl. Outra aposta no mesmo princípio: a spec como artefato primário.

Feche a promessa do S33: specs/ + AGENTS.md + pacote OKF no MESMO repositório. O que o sistema faz, como se trabalha nele e o domínio que ele modela — tudo markdown versionado.

Onde o SDD encontra os agentes



Contexto isolado por etapa, e o GATE em código DETERMINÍSTICO — não em outro modelo. Verificar com um segundo LLM só empilha incerteza; o portão tem que ser código que passa ou não passa.

Feche o S28: o monólito de 20 passos tem 36% de sucesso; a cadeia com gate se comporta de forma inteiramente diferente — o erro não se propaga.

É isto que vou mostrar rodando no Bloco 11: um gerador cujo portão determinístico pega o erro do agente ANTES do deploy.

O SDD entrega o artefato regulatório de graça

Software de saúde exige rastreabilidade requisito → projeto → teste → código. Mais gestão de risco documentada e ciclo de vida controlado: IEC 62304, ISO 14971 e a RDC da ANVISA.

Quem faz SDD JÁ TEM a matriz de rastreabilidade — como subproduto, versionada, com autoria e histórico.

A inversão da objeção: o SDD é o que torna código gerado por IA AUDITÁVEL. O que NÃO é auditável é código escrito à mão sem especificação.

Antipadrões: como o SDD morre na prática

Spec inflada. Escreve o que não vai verificar. Antídoto: só entra o que tem critério de aceitação.

Spec envelhecida. Código andou, spec não. Antídoto: CI que falha quando o código diverge da spec.

Gate humano virado carimbo. Aprova sem ler. Antídoto: o revisor assina o diff, não o PDF.

Spec gerada por IA e aprovada sem leitura — a PIOR. Produz aparência de rastreabilidade sem substância. Se ninguém leu, não há SDD, há teatro.

A escada da adaptação

Os degraus, do mais barato ao mais caro: prompt → poucos exemplos (few-shot) → RAG → ajuste fino / LoRA → aprendizado por reforço → pré-treino contínuo.

Só suba um degrau quando o anterior falhar COM EVIDÊNCIA MEDIDA. E a evidência é o conjunto de avaliação do S20 — não o “achismo”.

Retome a seta do S4: é aqui que se faz “o transplante em cima do modelo pronto” que prometi no começo.

Ajuste fino (SFT) e os dados

```
● ● ●  
{ "messages": [  
  { "role": "system",  
    "content": "Você classifica critérios de IRAS." },  
  { "role": "user",  
    "content": "<evolução clínica...>" },  
  { "role": "assistant",  
    "content": "{critério: ..., atendido: true}" }  
]}
```

Ajuste fino supervisionado = ensinar por exemplos rotulados. O formato é JSONL: cada linha é uma conversa completa (sistema, usuário, resposta).

500 a 5.000 exemplos BEM curados superam 100 mil sujos. Qualidade vence quantidade — de longe.

O custo real de um dataset clínico é tempo de infectologista, não GPU.

LoRA e QLoRA: ajuste fino que cabe numa GPU

```
LoraConfig(  
    r=16,                # posto: quão "grande" é o remendo  
    lora_alpha=32,  
    lora_dropout=0.05,  
    target_modules=["q_proj", "k_proj",  
                   "v_proj", "o_proj"],  
    task_type="CAUSAL_LM")
```

A ideia: congela o modelo W e treina só um **remendo pequeno** ($\Delta W = B \cdot A$). Treina 0,1–1% dos parâmetros — daí caber numa GPU de consumo.

QLoRA: base em 4 bits + **adaptadores em precisão maior**. É o que permite ajustar um modelo grande em 24 GB.

Vantagem operacional: adaptadores são arquivos de dezenas de MB — **versionáveis e trocáveis em runtime**. Um adaptador por especialidade, a MESMA base (pode ser o Qwen3.8-27B do S12).

RLHF e sucessores

RLHF = aprendizado por reforço com feedback humano. Comparações humanas → modelo de recompensa → otimização por reforço (PPO).

Sucessores: DPO (sem modelo de recompensa), GRPO (raciocínio), RLAIIF / IA Constitucional.

Por que importa mesmo sem treinar: explica o comportamento do modelo que vocês CONSOMEM.
Bajulação, recusa e excesso de ressalvas são artefatos do alinhamento — não do pré-treino.

Quando NÃO treinar, e o modelo local

Não treine com conhecimento volátil (protocolo muda → use RAG), tarefa que muda toda semana, ou sem avaliação. Custos ocultos: esquecimento catastrófico e manutenção perpétua do modelo derivado.

O modelo LOCAL (retomando o S12): quantização (GGUF, AWQ) + runtimes (LM Studio, vLLM, Ollama).

Três argumentos que fecham a palestra técnica: o dado do paciente não sai da rede · custo marginal previsível · independência de fornecedor.

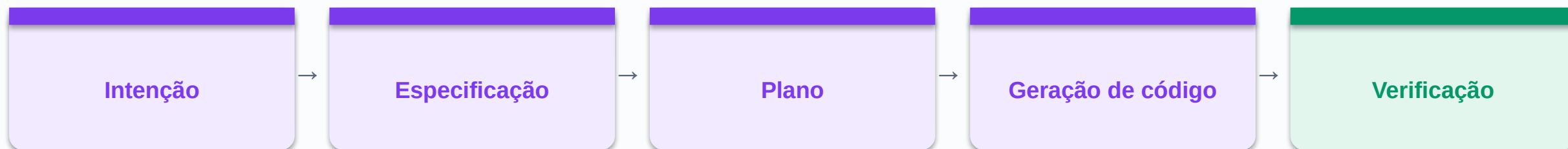
Ponte: o Co-Scientist e o domínio de vocês

O AI Co-Scientist da Google foi validado em laboratório (Nature, 2026) em três cenários. Repurposing de fármaco para leucemia · alvos para fibrose hepática · e um mecanismo de RESISTÊNCIA ANTIMICROBIANA.

O caso de RAM, nas palavras deles: com pouca informação de fundo, o sistema propôs — sozinho — que os cf-PICIs carregam genes de resistência entre espécies, incluindo E. coli e K. pneumoniae.

Isso recapitulou um achado experimental AINDA NÃO PUBLICADO — uma década de bancada. O segundo sistema que vou mostrar foi validado, na origem, exatamente no problema de vocês.

LangNet: arquitetura



Modelo local no centro: Qwen2.5-Coder-32B via LM Studio, contexto de 64 mil tokens; framework CrewAI. Com avaliação em curso do Qwen3.8-27B — por visão, uso de ferramentas e raciocínio, sem perder geração de código.

Amarre com o S12: o modelo em avaliação roda em 24 GB e é o melhor denso multimodal da faixa. O argumento inteiro do LangNet só existe porque essa faixa chegou a esse patamar em 2026.

LangNet: evidência medida

Cobertura de rastreabilidade: 100%. Todos os requisitos funcionais atravessam Spec → Modelo de Dados → Implementação, medido por PORTÃO DETERMINÍSTICO — não por confiança no modelo.

Suíte de tarefas geradas: 100% executando ponta a ponta. Contra banco real — núcleo determinístico + tarefas com agente (laudo, extração de documento).

~10 defeitos do gerador capturados pelo portão ANTES do deploy. Variável indefinida, coluna inexistente, junção espacial faltante — cada um com o salto exato onde quebrava.

O número não é a taxa de acerto do modelo — é a taxa em que o portão **pega o erro antes de virar produção**. É o S50 acontecendo. (Remeça no exemplo clínico do vídeo.)

DEMONSTRAÇÃO GRAVADA: da spec à aplicação

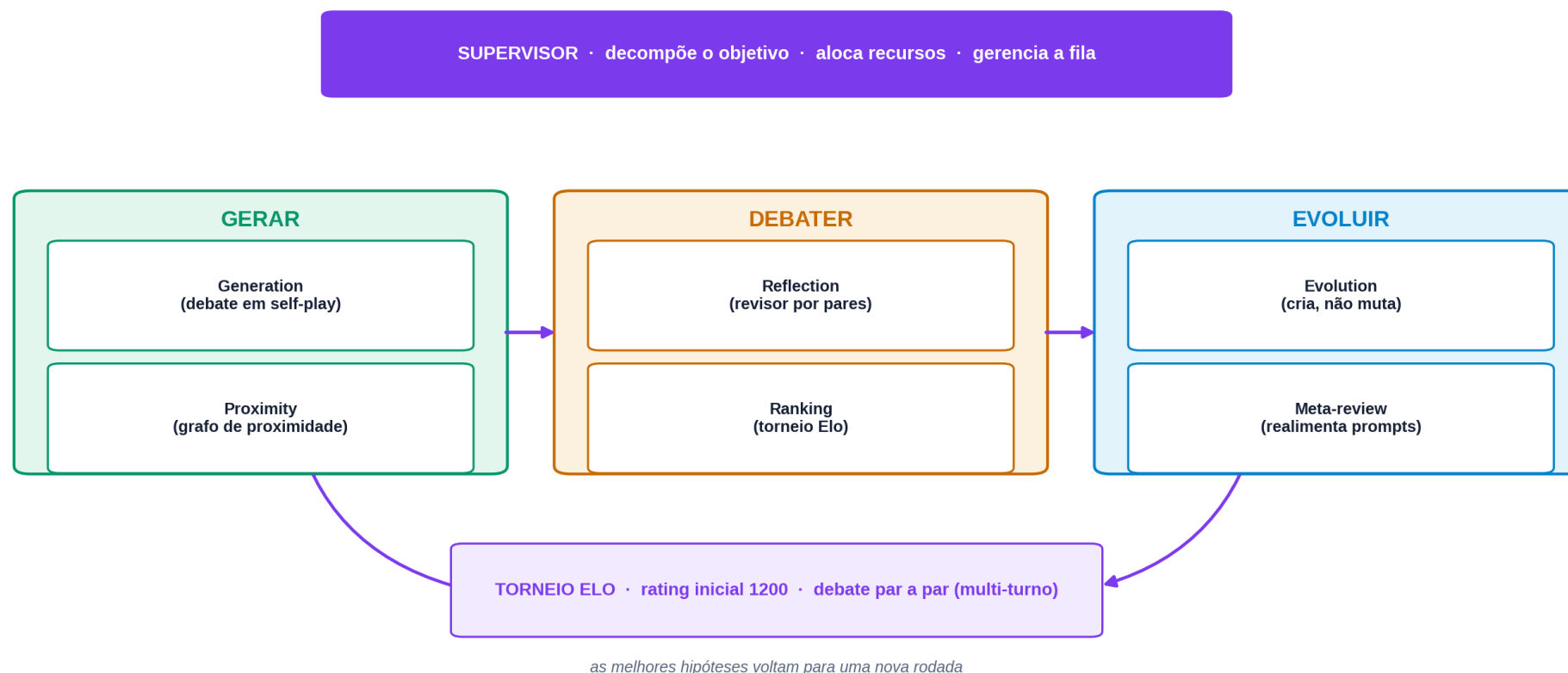
▶ VÍDEO — até 3 min, sem áudio, legendas grandes



(inserir a gravação da tela do LangNet)

- 1. você escreve uma spec curta (15–20 linhas) — deixe na tela para ler
- 2. geração do plano e das tarefas
- 3. app completo: modelo de dados, lógica, API e **interface**
- 4. execução da suíte derivada dos critérios
- 5. uma falha de teste e a correção — **não corte**
- 6. a aplicação rodando com a interface
- 7. volta à spec, com o **portão de rastreabilidade VERDE**

Como funciona o AI Co-Scientist (Nature, 2026)



Test-time compute: quanto mais “pensa”, maior o Elo. Meta-revisão = aprendizado SEM gradiente (em contexto).

Base: Gemini 2.0 · validação: Elo mais alto ↔ acerto no GPQA (AUC 0,643 → 0,651)

A nossa replicação: QuanticaResearch

O que MANTIVEMOS do paper (no código). Os 6 agentes + Supervisor em LangGraph; torneio Elo (nota inicial 1200, K=32), debate multi-turno guiado pelo grafo de proximidade; meta-revisão realimentando os prompts.

O que MUDAMOS, e por quê (aqui está a engenharia). LLM agnóstico de fornecedor — inclusive LOCAL (Ollama, LM Studio) — no lugar do “só Gemini”; fila durável própria (FastAPI + MariaDB) no lugar do agendador do Google; grafo de proximidade com NetworkX + Louvain.

Resultado exercitado (run UC-10). Rodou ponta a ponta: revisão de literatura + 4 hipóteses + 1º torneio + 1ª meta-revisão, em ~2,8 h com DeepSeek.

Honestidade: réplica funcional PRONTA PARA PILOTO, não produção. A sincronização com o banco ainda tem um bug conhecido (alfa).

A camada que amarra tudo: Redes de Petri

VisualTasksExec: sincroniza agentes com propriedades VERIFICÁVEIS. Ausência de deadlock (travamento), alcançabilidade, invariantes de estado — provados na estrutura, não testados por amostragem.

Feche o arco inteiro da palestra: o S28 mostrou que agentes degradam de forma previsível; o S50, que gates cortam a cadeia; o S30, que nenhum protocolo expressa governança; o S62, um sistema que se autoavalia por Elo.

Mas o Elo mede qualidade de HIPÓTESE, não corretude de EXECUÇÃO. As Redes de Petri mostram que a estrutura da orquestração pode ser VERIFICADA FORMALMENTE — não apenas testada ou pontuada.

As três conclusões

① **A capacidade virou mercadoria comum; o método, não.** O que separa os times é especificar e verificar — não o acesso ao modelo.

② **Agente sem portão degrada de forma previsível.** Confiabilidade vem de reduzir passos não verificados (S28, S50).

③ **A especificação executável é o que torna código de IA auditável.** E, em software de saúde, entrega o artefato regulatório de graça (S51).

Recomendação acionável, uma só: construam PRIMEIRO o conjunto de avaliação (S20). Antes de framework, antes de agente, antes de trocar de modelo.

Referências e contato

Papers e specs. Co-Scientist: Gottweis et al., Nature 2026 (s41586-026-10644-y) / arXiv 2502.18864. Specs: MCP, A2A, OKF, GitHub Spec Kit.

Modelos. A tabela do S12, com os links — reconferida na semana da palestra.

Repositórios e contato. QR do repositório do LangNet e do QuanticaResearch; seu e-mail e perfil.

Perguntas

“O gargalo deixou de ser escrever código. Passou a ser especificar e verificar.”

Obrigado.